Dataframes with pandas

Introduction to pandas

Pandas is fully operational on its own. Pandas' key functionality is the manipulation and analysis of tabular data

Tabular data

- Data that is in the form of a table, with rows and columns
- A spreadsheet is a common example of tabular data

While NumPy is capable of many of the same functions and operations as pandas,

Pandas

- Pandas, on the other hand, provides a simple interface that allows you to display your data as rows and columns
- This means that you can always follow exactly what's happening to your data as you manipulate it.

Data frame

- a core data structure in pandas

pandas basics

Core pandas object classes

- dataframe
    - A two-dimensional, labeled data structure with rows and columns
- series

DataFrame()

We can create a dataframe using the pandas DataFrame function. This function has a lot of flexibility, and can convert numerous data formats to a DataFrame

object.

```
import pandas as pd

data = {
   "PassengerId": [1, 2, 3, 4, 5, 6],
   "Survived": [0, 1, 1, 1, 0, 0],
   "Pclass": [3, 1, 3, 1, 3, 3],
   "Name": [
      "Braund, Mr. Owen Harris",
      "Cumings, Mrs. John Bradley (Florence Briggs Thayer)",
      "Heikkinen, Miss. Laina",
      "Futrelle, Mrs. Jacques Heath (Lily May Peel)",
      "Allen, Mr. William Henry",
      "Moran, Mr. James"
   ],
   "Sex": ["male", "female", "female", "female", "male", "male"],
   "Age": [22.0, 38.0, 26.0, 35.0, 35.0, None],
   "SibSp": [1, 1, 0, 1, 0, 0],
   "Parch": [0, 0, 0, 0, 0, 0],
   "Ticket": ["A/5 21171", "PC 17599", "STON/O2. 3101282", "113803", "373450
   "Fare": [7.25, 71.2833, 7.925, 53.1, 8.05, 8.4583],
   "Cabin": [None, "C85", None, "C123", None, None],
   "Embarked": ["S", "C", "S", "S", "S", "Q"]
}

df
```

```
import numpy as np
import pandas as pd

df2 = pd.DataFrame(np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]]),
                   columns=['a', 'b', 'c'], index=['x', 'y', 'z'])
df2
```

	a	b	c
x	1	2	3

y	4	5	6
z	7	8	9

These are just a couple of the many different ways to create a dataframe with the DataFrame function

read_csv()

assume that we have a data.csv file like this:

```
name,age,city
Alice,30,New York
Bob,25,Los Angeles
Charlie,35,Chicago
```

```
import pandas as pd

# Read the CSV file
df = pd.read_csv('data.csv')

# Display the DataFrame
print(df)
```

```
output:
      name  age         city
0    Alice   30     New York
1      Bob   25  Los Angeles
2  Charlie   35      Chicago
```

Series

- A one-dimensional, labeled array
- Series objects are most often used to represent individual columns or rows of a dataframe.

Work with dataframe

```
titanic.columns
→ return columns
titanic.shape
→ return (number of rows, number of columns)
titanic.info()
#we can get some summary information about the dataframe by calling the in
```

NaN

- How null values are represented in pandas, which stands for “not a number”

if you want to select a single column, you can type the name of the dataframe followed by brackets, and within the brackets enter the name of the column as a string. This returns a Series object of that column.

```
titanic["Age"]
#same as
titanic.Age
```

You can also use dot notation, but this only works if the column name does not contain any whitespaces.

Using dot notation is faster to type, so for very simple lines of code, you may prefer to do this. But if the code begins to get more complex, it's generally better to use bracket notation, because it makes the code easier to read.

To select multiple columns of a dataframe by name, use bracket notation. Within the brackets, enter a list of column names.

This returns a view of your dataframe as a new DataFrame object.

```
titanic[["Name", "Age"]]
```

iloc[]

- If you want to select rows or columns by index, you'll need to use iloc.
- A way to indicate in pandas that you want to select by interger-location-based position

If you enter a single integer into the iloc brackets, you'll get a series object representing a single row of your dataframe at that index.

```
titanic.iloc[0]
```

Because I entered 0 here, I got the very first row in my dataframe as a series.

```
'''
If you enter a LIST of a single integer in the iloc brackets,
you'll get a DataFrame object of a single row of the dataframe at that index.
'''
titanic.iloc[[0, 3]]
'''
You can access a range of rows by entering the indices of the beginning
and ending rows separated by a colon.
Pandas will return every index starting with the beginning index up to,
but not including, the last index. So zero colon three returns row indices 0, 1, a
'''
#or
titanic.iloc[0:3]
```

You can select subsets of rows and columns together, too.

```
titanic.iloc[0:3, [3, 4]]
#This returns a dataframe view of rows 0, 1, and 2 at columns 3 and 4 only.
```

if you want a single column in its entirety, you select all rows, and then enter the index of the column you want

```
titanic.iloc[:, [3,4]]
```

And, you can even get a single value at a particular row in a particular column by using two indices separated by a comma.

```
titanic.iloc[3,4]
```

loc[]

- used to select pandas rows and columns by name
- Loc is similar to iloc, but instead of selecting by index location, loc is used to select pandas rows and columns by name.

```
titanic.loc[1:3, ['Name']]
# select row 1 to 3 with Name column
```

If you want to add a new column to a dataframe, you can do that with a simple assignment statement.

```
titanic["age_add_100"] = titanic["Age"] + 100
```

The fundamentals of pandas

You've learned that Python has many open-source libraries and packages—including NumPy and pandas—that make it one of the most useful coding languages. In this reading, you will review the basics of pandas dataframes and learn more about how to work with them. Understanding the fundamentals of pandas is essential to becoming a capable and competent data professional.

Primary data structures

Pandas has two primary data structures: **Series** and **DataFrame**.

- **Series:** A Series is a one-dimensional labeled array that can hold any data type. It's similar to a column in a spreadsheet or a one-dimensional NumPy array. Each element in a series has an associated label called an index. The index allows for more efficient and intuitive data manipulation by making it easier to reference specific elements of your data.
- **DataFrame:** A dataframe is a two-dimensional labeled data structure—essentially a table or spreadsheet—where each column and row is represented by a Series.

Create a DataFrame

To use pandas in your notebook, first import it. Similar to NumPy, pandas has its own standard alias, **pd**, that's used by data professionals around the world:

```
import pandas as pd
```

Once you've imported pandas into your working environment, create a dataframe. Here are some of the ways to create a **DataFrame** object in a Jupyter Notebook.

From a dictionary:

```
d = {'col1': [1, 2], 'col2': [3, 4]}
df = pd.DataFrame(data=d)
df

##
   col1  col2
0     1     3
1     2     4
```

From a numpy array:

```
df2 = pd.DataFrame(np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]]),
                   columns=['a', 'b', 'c'])
df2

   a  b  c
0  1  2  3
1  4  5  6
2  7  8  9
```

From a comma-separated values (csv) file:

(Note that this cell will not run, but is provided to illustrate syntax.)

```
df3 = pd.read_csv('/file_path/file_name.csv')
```

Attributes and methods

The **DataFrame** class is powerful and convenient because it comes with a suite of built-in features that simplify common data analysis tasks. These features are known as attributes and methods. An attribute is a value associated with an

object or class that is referenced by name using dotted expressions. A method is a function that is defined inside a class body and typically performs an action. A simpler way of thinking about the distinction between attributes and methods is to remember that attributes are *characteristics* of the object, while methods are *actions* or *operations*.

Common DataFrame attributes

Data professionals use attributes and methods constantly. Some of the most-used **DataFrame** attributes include:

Attribute	Description
columns	Returns the column labels of the dataframe
dtypes	Returns the data types in the dataframe
iloc	Accesses a group of rows and columns using integer-based indexing
loc	Accesses a group of rows and columns by label(s) or a Boolean array
shape	Returns a tuple representing the dimensionality of the dataframe
values	Returns a NumPy representation of the dataframe

Common DataFrame methods

Some of the most-used **DataFrame** methods include:

Method	Description
apply()	Applies a function over an axis of the dataframe
copy()	Makes a copy of the dataframe's indices and data
describe()	Returns descriptive statistics of the dataframe, including the minimum, maximum, mean, and percentile values of its numeric columns; the row count; and the data types
drop()	Drops specified labels from rows or columns
groupby()	Splits the dataframe, applies a function, and combines the results
head(*n=5*)	Returns the first *n* rows of the dataframe (default=5)
info()	Returns a concise summary of the dataframe
isna()	Returns a same-sized Boolean dataframe indicating whether each value is null (can also use **isnull()** as an alias)
sort_values()	Sorts by the values across a given axis
value_counts()	Returns a series containing counts of unique rows in the dataframe
where()	Replaces values in the dataframe where a given condition is false

These are just a handful of some of the most commonly used attributes and methods—there are many, many more! Some of them can also be used on pandas **Series** objects. For a more detailed list, refer to the pandas DataFrame documentation, which includes helpful examples of how to use each tool.

Selection statements

Once your data is read into a dataframe, you'll want to do things with it by selecting, manipulating, and evaluating the data. In this section, you'll learn how to select rows, columns, combinations of rows and columns, and basic subsets of data.

Row selection

Rows of a dataframe are selected by their index. The index can be referenced either by name or by numeric position.

loc[]

loc[] lets you select rows by name. Here's an example:

```
df = pd.DataFrame({
  'A': ['alpha', 'apple', 'arsenic', 'angel', 'android'],
  'B': [1, 2, 3, 4, 5],
  'C': ['coconut', 'curse', 'cassava', 'cuckoo', 'clarinet'],
  'D': [6, 7, 8, 9, 10]
  },
  index=['row_0', 'row_1', 'row_2', 'row_3', 'row_4'])
df

##
          A  B        C   D
row_0    alpha  1   coconut   6
row_1    apple  2     curse   7
row_2  arsenic  3   cassava   8
row_3    angel  4    cuckoo   9
row_4  android  5  clarinet  10
```

The row index of the dataframe contains the names of the rows. Use **loc[]** to select rows by name:

```
print(df.loc['row_1'])

A    apple
B        2
C    curse
D        7
Name: row_1, dtype: object
```

Inserting just the row index name in selector brackets returns a **Series** object. Inserting the row index name as a list returns a **DataFrame** object:

```
print(df.loc[['row_1']])

         A  B      C  D
row_1  apple  2  curse  7
```

To select multiple rows by name, use a list within selector brackets:

```
print(df.loc[['row_2', 'row_4']])

##
          A  B        C   D
row_2  arsenic  3   cassava   8
row_4  android  5  clarinet  10
```

You can even specify a range of rows by named index:

```
print(df.loc['row_0':'row_3'])

           A  B       C  D
row_0    alpha  1  coconut  6
row_1    apple  2    curse  7
row_2  arsenic  3  cassava  8
row_3    angel  4   cuckoo  9
```

Note: Because you're using named indices, the returned range includes the specified end index.

iloc[]

iloc[] lets you select rows by numeric position, similar to how you would access elements of a list or an array. Here's an example.

```
print(df)
print()
print(df.iloc[1])

          A  B        C   D
row_0    alpha  1   coconut   6
row_1    apple  2     curse   7
row_2  arsenic  3   cassava   8
row_3    angel  4    cuckoo   9
row_4  android  5  clarinet  10

A    apple
B        2
C    curse
D        7
Name: row_1, dtype: object
```

Inserting just the row index number in selector brackets returns a **Series** object. Inserting the row index number as a list returns a **DataFrame** object:

```
print(df.iloc[[1]])

#
        A  B      C  D
row_1  apple  2  curse  7
```

To select multiple rows by index number, use a list within selector brackets:

```
print(df.iloc[[0, 2, 4]])

##
          A  B        C   D
row_0    alpha  1   coconut   6
```

```
row_2  arsenic  3   cassava   8
row_4  android  5  clarinet  10
```

Specify a range of rows by index number:

```
print(df.iloc[0:3])

          A  B        C  D
row_0    alpha  1  coconut  6
row_1    apple  2    curse  7
row_2  arsenic  3  cassava  8
```

Note that this does not include the row at index three.

Column selection

Bracket notation

Column selection works the same way as row selection, but there are also some shortcuts to make the process easier. For example, to select an individual column, simply put it in selector brackets after the name of the dataframe:

```
print(df['C'])

row_0     coconut
row_1      curse
row_2     cassava
row_3      cuckoo
row_4    clarinet
Name: C, dtype: object
```

And to select multiple columns, use a list in selector brackets:

```
print(df[['A', 'C']])

          A         C
row_0    alpha   coconut
row_1    apple     curse
row_2  arsenic   cassava
```

```
row_3    angel    cuckoo
row_4  android  clarinet
```

Dot notation

It’s possible to select columns using dot notation instead of bracket notation. For example:

```
print(df.A)

row_0      alpha
row_1      apple
row_2    arsenic
row_3      angel
row_4    android
Name: A, dtype: object
```

Dot notation is often convenient and easier to type. However, it can make your code more difficult to read, especially in longer statements involving method chaining or condition-based selection. For this reason, bracket notation is often preferred.

loc[]

You can also use **loc[]** notation:

```
print(df)
print()

print(df.loc[:, ['B', 'D']])

          A  B        C   D
row_0    alpha  1   coconut   6
row_1    apple  2     curse   7
row_2  arsenic  3   cassava   8
row_3    angel  4    cuckoo   9
row_4  android  5  clarinet  10

      B   D
```

```
row_0  1   6
row_1  2   7
row_2  3   8
row_3  4   9
row_4  5  10
```

Note that when using **loc[]** to select columns, you must specify rows as well. In this example, all rows were selected using just a colon (**:**).

iloc[]

Similarly, you can use **iloc[]** notation. Again, when using **iloc[]**, you must specify rows, even if you want to select all rows:

```
print(df.iloc[:, [1,3]])

##
      B   D
row_0  1   6
row_1  2   7
row_2  3   8
row_3  4   9
row_4  5  10
```

Select rows and columns

Both **loc[]** and **iloc[]** can be used to select specific rows and columns together.

loc[]

```
print(df.loc['row_0':'row_2', ['A','C']])

##
          A        C
row_0    alpha  coconut
row_1    apple    curse
row_2  arsenic  cassava
```

Again, notice that when using **loc[]** to select a range, the final element in the range is included in the results.

iloc[]

```
print(df.iloc[[2, 4], 0:3])

##
         A  B        C
row_2  arsenic  3   cassava
row_4  android  5  clarinet
```

Note that, when using rows with named indices, you cannot mix numeric and named notation. For example, the following code will throw an error:

```
print(df.loc[0:3, ['D']])

##
Error on line 1:
   print(df.loc[0:3, ['D']])
```

To view rows **[0:3]** at column '**D**' (if you don't know the index number of column D), you'd have to use selector brackets after an **iloc[]** statement:

```
# This is most convenient for VIEWING:
print(df.iloc[0:3][['D']])

# But this is best practice/more stable for assignment/manipulation:
print(df.loc[df.index[0:3], 'D'])


#####
      D
row_0  6
row_1  7
row_2  8
row_0    6
row_1    7
```

```
row_2    8
Name: D, dtype: int64
```

However, in many (perhaps most) cases your rows will not have named indices, but rather numeric indices. In this case, you can mix numeric and named notation. For example, here's the same dataset, but with numeric indices instead of named indices.

```
df = pd.DataFrame({
  'A': ['alpha', 'apple', 'arsenic', 'angel', 'android'],
  'B': [1, 2, 3, 4, 5],
  'C': ['coconut', 'curse', 'cassava', 'cuckoo', 'clarinet'],
  'D': [6, 7, 8, 9, 10]
  },
  )
df

##
       A  B        C   D
0    alpha  1   coconut   6
1    apple  2     curse   7
2  arsenic  3   cassava   8
3    angel  4    cuckoo   9
4  android  5  clarinet  10
```

Notice that the rows are enumerated now. Now, this code will execute without error:

```
print(df.loc[0:3, ['D']])

##
  D
0  6
1  7
2  8
3  9
```

Key takeaways

Pandas dataframes are a convenient way to work with tabular data. Each row and each column can be represented by a pandas **Series**, which is similar to a one-dimensional array. Both dataframes and series have a large collection of methods and attributes to perform common tasks and retrieve information. Pandas also has its own special notation to select data. As you work more with pandas, you'll become more comfortable with this notation and its many applications in data science.

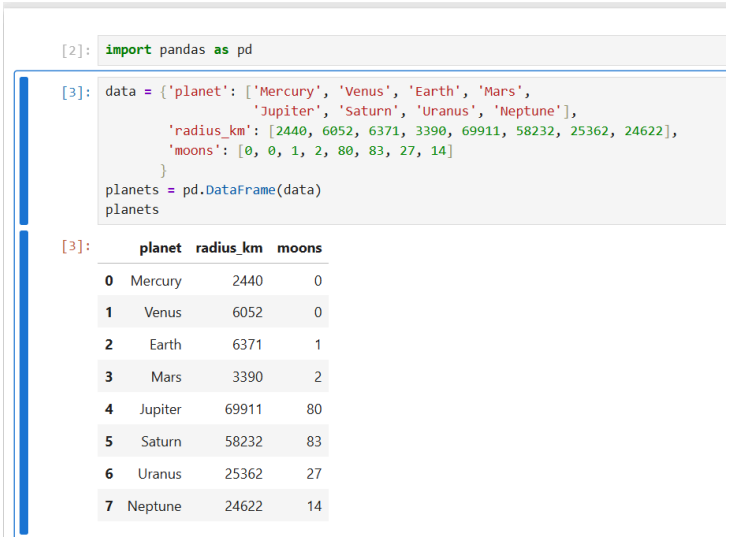
Resources for more information

- pandas DataFrame class documentation
- pandas Series class documentation
- pandas selection documentation

Boolean masking

- A filtering technique that overlays a Boolean grid onto a dataframe in order to select only the values in the dataframe that align with the True values of the grid

Moons < 20?		Planet	radius_km	Moons
True	0	Mercury	2,440	0
True	1	Venus	6,052	0
True	2	Earth	6,371	1
True	3	Mars	3,390	2
False	4	Jupiter	69,911	80
False	5	Saturn	58,232	83
False	6	Uranus	25,362	27
True	7	Neptune	24,622	14

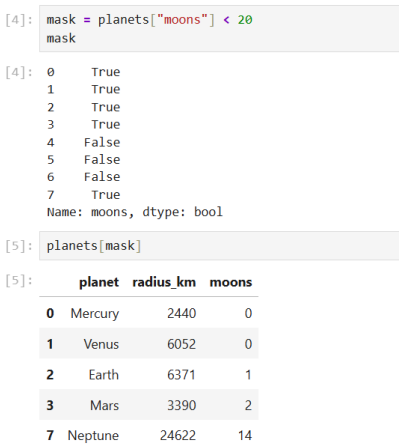


```
[2]: import pandas as pd

[3]: data = {'planet': ['Mercury', 'Venus', 'Earth', 'Mars',
                        'Jupiter', 'Saturn', 'Uranus', 'Neptune'],
             'radius_km': [2440, 6052, 6371, 3390, 69911, 58232, 25362, 24622],
             'moons': [0, 0, 1, 2, 80, 83, 27, 14]
            }
     planets = pd.DataFrame(data)
     planets
```

[3]:

	planet	radius_km	moons
0	Mercury	2440	0
1	Venus	6052	0
2	Earth	6371	1
3	Mars	3390	2
4	Jupiter	69911	80
5	Saturn	58232	83
6	Uranus	25362	27
7	Neptune	24622	14



```
[4]: mask = planets["moons"] < 20
     mask

[4]: 0     True
     1     True
     2     True
     3     True
     4    False
     5    False
     6    False
     7     True
     Name: moons, dtype: bool

[5]: planets[mask]
```

[5]:

	planet	radius_km	moons
0	Mercury	2440	0
1	Venus	6052	0
2	Earth	6371	1
3	Mars	3390	2
7	Neptune	24622	14

- Note that we haven't permanently modified the dataframe. Applying the boolean mask using the conditional logic only gives a filtered "view" of it.
- So, when you call the planet's variable again it returns the full dataframe.
- However, we can assign the result to a named variable. This may be useful if you'll need to reference the list of planets with moons under 20 again later.

operator and logic

operator	logic
&	and
|	or
~	not

eg:

```
mask = (planets["moons"] < 10 | planets["moons"] >50)

# must have more than 20 moons
# must not have 80 moons
# must not have a radius less than 50.000km
mask = (planets["moons"]  >20 & ~(planets["moons"] == 80) & ~(planets["rad
```

→ maybe same as a query with condition in SQL

Boolean masking in pandas

Now that you know how to select data in pandas by referring to rows and columns, the next step is to learn how to use Boolean masks. Data professionals use Boolean masks to select data in pandas based on conditions. In this reading, you will discover Boolean masking and how to use pandas' logical operators to form multi-conditional selection statements. Understanding the fundamentals of pandas will help make your work as a data professional easier and more efficient.

Boolean masks

You know that Boolean is used to describe any binary variable whose possible values are true or false. With pandas, **Boolean masking,** also called **Boolean indexing**, is used to overlay a Boolean grid onto a dataframe's index in order to select only the values in the dataframe that align with the **True** values of the grid.

Return to the example from the video. Suppose you have a dataframe of planets, their radii, and their number of moons:

planet	radius_km	moons
Mercury	2,440	0
Venus	6,052	0
Earth	6,371	1
Mars	3,390	2
Jupiter	69,911	80
Saturn	58,232	83
Uranus	25,362	27
Neptune	24,622	14

Now suppose that you want to keep the rows of any planets that have fewer than 20 moons and filter out the rest. A Boolean mask is a pandas **Series** object indicating whether this condition is true or false for each value in the **moons** column:

	Moons < 20?
0	**True**
1	**True**

2	**True**
3	**True**
4	**False**
5	**False**
6	**False**
7	**True**

The **dtype** contained in this series is **bool**. Boolean masking effectively overlays this Boolean series onto the dataframe's index. The result is that any rows in the dataframe that are indicated as **False** in the Boolean mask get filtered out, and any rows that are indicated as **True** remain in the dataframe:

planet	radius_km	moons
Mercury	2,440	0
Venus	6,052	0
Earth	6,371	1
Mars	3,390	2
Neptune	24,622	14

Coding Boolean masks in pandas

Here is how to perform this operation in pandas.

Begin with a **DataFrame** object.

```
data = {'planet': ['Mercury', 'Venus', 'Earth', 'Mars',
                   'Jupiter', 'Saturn', 'Uranus', 'Neptune'],
        'radius_km': [2440, 6052, 6371, 3390, 69911, 58232,
                      25362, 24622],
        'moons': [0, 0, 1, 2, 80, 83, 27, 14]
        }
df = pd.DataFrame(data)
df

##
  moons   planet  radius_km
0     0  Mercury       2440
1     0    Venus       6052
```

```
2     1    Earth       6371
3     2     Mars       3390
4    80  Jupiter      69911
5    83   Saturn      58232
6    27   Uranus      25362
7    14  Neptune      24622
```

Then, write a logical statement. Remember, the objective is to keep planets that have fewer than 20 moons and filter out the rest.

```
print(df['moons'] < 20)

0     True
1     True
2     True
3     True
4    False
5    False
6    False
7     True
Name: moons, dtype: bool
```

This results in a **Series** object of **dtype: bool** that consists of the row indices, where each index contains a **True** or **False** value depending on whether that row satisfies the given condition. This is the Boolean mask. To apply this mask to the dataframe, simply insert this statement into selector brackets and apply it to your dataframe:

```
print(df[df['moons'] < 20])
##
   moons   planet  radius_km
0      0  Mercury       2440
1      0    Venus       6052
2      1    Earth       6371
3      2     Mars       3390
7     14  Neptune      24622
```

You can also assign the Boolean mask to a named variable and then apply that to your dataframe:

```
mask = df['moons'] < 20
df[mask]

##
  moons   planet  radius_km
0     0  Mercury       2440
1     0    Venus       6052
2     1    Earth       6371
3     2     Mars       3390
7    14  Neptune      24622
```

Note that this doesn't permanently modify your dataframe. It only gives a filtered view of it.

However, you can assign the result to a named variable:

```
mask = df['moons'] < 20
df2 = df[mask]
df2

##
  moons   planet  radius_km
0     0  Mercury       2440
1     0    Venus       6052
2     1    Earth       6371
3     2     Mars       3390
7    14  Neptune      24622
```

And if you want to select just the planet column as a **Series** object, you can use regular selection tools like **loc[]**:

```
mask = df['moons'] < 20
df.loc[mask, 'planet']
##
0    Mercury
1      Venus
```

```
2     Earth
3      Mars
7   Neptune
Name: planet, dtype: object
```

Complex logical statements

In statements that use multiple conditions, pandas uses logical operators to indicate which data to keep and which to filter out. These operators are:

Operator	Logic
&	**and**
|	**or**
~	**not**

Important: Each component of a multi-conditional logical statement must be in parentheses. Otherwise, the statement will throw an error or, worse, return something that isn't what you intended.

For example, here is how to create a Boolean mask that selects all planets that have fewer than 10 moons or greater than 50 moons:

```
mask = (df['moons'] < 10) | (df['moons'] > 50)
mask
```

Key takeaways

A Boolean mask is a method of applying a filter to a dataframe. The mask overlays a Boolean grid over your dataframe in order to select only the values in the dataframe that align with the True values of the grid. To create Boolean comparisons, pandas has its own logical operators. These operators are:

- **&** (and)
- **|** (or)
- **~** (not)

Each criterion of a multi-conditional selection statement must be enclosed in its own set of parentheses. With practice, making complex selection statements in pandas is possible and efficient.

Resources for more information

- pandas Boolean indexing documentation

Grouping and aggregation

groupby()

- A pandas DataFrame method that groups rows of the dataframe together based on their values at one or more columns, which allows further analysis of the groups

Note: Beginning in pandas v.1.5.0, numerical aggregation functions like **sum()**, **mean()**, **median()**, etc. must have their **numeric_only** keyword argument set to **True** when used with a **groupby()** operation or else pandas will throw an error if your data also contains non-numerical columns.

For example, the operation demonstrated here would be:

```
planets.groupby(['type']).sum(numeric_only=True)
```

Groupby will work on multiple columns too.

```
planets.groupby(['type', 'magnetic_field']).sum(numeric_only=True)
```

Another important method to use on the groupby objects is the agg method.

agg()

- Short for “aggregate” A pandas groupby method that allows you to apply multiple calculations to groups of data

More on grouping and aggregation

You’ve discovered that pandas is a Python library that facilitates reviewing and manipulating tabular data. In addition, **groupby()** and **agg()** are essential **DataFrame** methods that data professionals use to group, aggregate, summarize, and better understand data. In this reading, you’ll review how these functions work, as well as when and how to apply them.

groupby()

The groupby() function is a method that belongs to the **DataFrame** class. It works by splitting data into groups based on specified criteria, applying a function to each group independently, then combining the results into a data structure. When applied to a dataframe, the function returns a groupby object. This groupby object serves as the foundation for different data manipulation operations, including:

- Aggregation: Computing summary statistics for each group
- Transformation: Applying functions to each group and returning modified data
- Filtration: Selecting specific groups based on certain conditions
- Iteration: Iterating over groups or values

Here are some examples that use the **groupby()** function on a dataframe consisting of different articles of clothing:

```
clothes = pd.DataFrame({'type': ['pants', 'shirt', 'shirt', 'pants', 'shirt', 'pants'],
                        'color': ['red', 'blue', 'green', 'blue', 'green', 'red'],
                        'price_usd': [20, 35, 50, 40, 100, 75],
                        'mass_g': [125, 440, 680, 200, 395, 485]})



clothes

##
  color  mass_g  price_usd   type
0   red    125         20  pants
1  blue    440         35  shirt
2 green    680         50  shirt
3  blue    200         40  pants
4 green    395        100  shirt
5   red    485         75  pants
```

Grouping the dataframe by **type** results in a **DataFrameGroupBy** object:

```
grouped = clothes.groupby('type')
print(grouped)
```

```
print(type(grouped))

##
<pandas.core.groupby.DataFrameGroupBy object at 0x7f6f5c2ee0f0>
<class 'pandas.core.groupby.DataFrameGroupBy'>
```

However, an aggregation function can be applied to the groupby object:

```
grouped = clothes.groupby('type')
grouped.mean()

##
      mass_g  price_usd
type
pants   270.0  45.000000
shirt   505.0  61.666667
```

In the preceding example, **groupby()** combined all the items into groups based on their type and returned a **DataFrame** object containing the mean of each group for each numeric column in the dataframe. Note: In future versions of pandas it will be necessary to specify a **numeric_only** parameter when applying certain aggregation functions—like mean—to a groupby object. **numeric_only** refers to the datatype of each column. In earlier versions of pandas (like the version on this platform) it isn't necessary to specify **numeric_only=True**, but in future versions this must be done. Otherwise, it will be necessary to indicate the specific columns to be captured.)

In addition, groups may be created based on multiple columns:

```
clothes.groupby(['type', 'color']).min()

##
            mass_g  price_usd
type  color
pants blue      200         40
      red       125         20
```

```
shirt blue     440       35
      green    395       50
```

In the preceding example, **groupby()** was called directly on the clothes dataframe. The data was grouped first by **type**, then by **color**. This resulted in four groups—the number of different existing combinations of values for type and color. Then, the **min()** function was applied to the result to filter each group by its minimum value.

To simply return the number of observations there are in each group, use the **size()** method. This will result in a **Series** object with the relevant information:

```
clothes.groupby(['type', 'color']).size()

##
type   color
pants  blue    1
       red     2
shirt  blue    1
       green   2
dtype: int64
```

Built-in aggregation functions

The previous examples demonstrated the **mean()**, **min()**, and **size()** aggregation functions applied to groupby objects. There are many available built-in aggregation functions. Some of the more commonly used include:

- **count()**: The number of non-null values in each group
- **sum()**: The sum of values in each group
- **mean()**: The mean of values in each group
- **median()**: The median of values in each group
- **min()**: The minimum value in each group
- **max()**: The maximum value in each group
- **std()**: The standard deviation of values in each group
- **var()**: The variance of values in each group

agg()

The agg() function is useful when you want to apply multiple functions to a dataframe at the same time. **agg()** is a method that belongs to the **DataFrame** class. It stands for "aggregate." Its most important parameters are:

- **func**: The function to be applied
- **axis**: The axis over which to apply the function (default= 0).

Following are some examples of how **agg()** can be used. Note that they demonstrate how this function can be used by itself (without **groupby()**). Note also that, due to platform limitations, some of the following code blocks are not executable. In these cases, output is provided as an image. Here is the original **clothes** dataframe again as a reminder:

```
clothes

  color  mass_g  price_usd   type
0   red     125         20  pants
1  blue     440         35  shirt
2 green     680         50  shirt
3  blue     200         40  pants
4 green     395        100  shirt
5   red     485         75  pants
```

The following example applies the **sum()** and **mean()** functions to the **price** and **mass_g** columns of the **clothes** dataframe.

```
clothes[['price_usd', 'mass_g']].agg(['sum', 'mean'])

##
       price_usd  mass_g
sum      320.000  2323.0
mean      53.333   387.5
```

Notice the following:

- The two columns are subset from the dataframe before applying the **agg()** method. If you don't subset the relevant columns first, **agg()** will attempt to

apply **sum()** and **mean()** to *all* of the columns, which wouldn't work because some columns contain strings. (Technically, **sum()** would work, but it would return something useless because it would just combine all the strings into one long string.)

- The **sum()** and **mean()** functions are entered as strings in a list, without their parentheses. This will work for any built-in aggregation function.

In this next example, different functions are applied to different columns.

```
clothes.agg({'price_usd': 'sum',
             'mass_g': ['mean', 'median']
             })



##
        price_usd  mass_g
sum   320.000       NaN
mean      NaN     387.5
median    NaN     417.5
```

Notice the following:

- Columns are not subset from the dataframe before applying the **agg()** function. This is unnecessary because the columns are specified within the **agg()** function itself.
- The argument to the **agg()** function is a dictionary whose keys are columns and whose values are the functions to be applied to those columns. If multiple functions are applied to a column, they are entered as a list. Again, each built-in function is entered as a string without parentheses.
- The resulting dataframe contains **NaN** values where a given function was not designated to be used.

The following example applies the **sum()** and **mean()** functions across axis 1. In other words, instead of applying the functions down each column, they're applied over each row.

```
clothes[['price_usd', 'mass_g']].agg(['sum', 'mean'], axis=1)
```

```
##
   sum   mean
0 145.0  72.5
1 475.0 237.5
2 730.0 365.0
3 240.0 120.0
4 495.0 247.5
5 560.0 280.0

clothes
   color  mass_g  price_usd   type
0    red     125         20  pants
1   blue     440         35  shirt
2  green     680         50  shirt
3   blue     200         40  pants
4  green     395        100  shirt
5    red     485         75  pants
```

groupby() with agg()

The **groupby()** and **agg()** functions are often used together. In such cases, first apply the **groupby()** function to a dataframe, then apply the **agg()** function to the result of the groupby. For reference, here is the **clothes** dataframe once again.

```
clothes

##
   color  mass_g  price_usd   type
0    red     125         20  pants
1   blue     440         35  shirt
2  green     680         50  shirt
3   blue     200         40  pants
4  green     395        100  shirt
5    red     485         75  pants
```

In the following example, the items in **clothes** are grouped by **color**, then each of those groups has the **mean()** and **max()** functions applied to them at the

price_usd and **mass_g** columns.

```
clothes.groupby('color').agg({'price_usd': ['mean', 'max'],
                              'mass_g': ['mean', 'max']})
```

```
      price_usd      mass_g
           mean  max   mean  max
color
blue       37.5   40  320.0  440
green      75.0  100  537.5  680
red        47.5   75  305.0  485
```

MultiIndex

You might have noticed that, when functions are applied to a groupby object, the resulting dataframe has tiered indices. This is an example of **MultiIndex**. MultiIndex is a hierarchical system of dataframe indexing. It enables you to store and manipulate data with any number of dimensions in lower dimensional data structures such as series and dataframes. This facilitates complex data manipulation.

This course will not require any deep knowledge of hierarchical indexing, but it's helpful to be familiar with it. Consider the following example:

```
grouped = clothes.groupby(['color', 'type']).agg(['mean', 'min'])
grouped

#
            mass_g      price_usd
              mean  min      mean min
color type
blue  pants  200.0  200      40.0  40
      shirt  440.0  440      35.0  35
green shirt  537.5  395      75.0  50
red   pants  305.0  125      47.5  20
```

Notice that **color** and **type** are positioned lower than the column names in the output. This indicates that **color** and **type** are no longer columns, but named row indices. Similarly, notice that **price_usd** and **mass_g** are positioned above **mean** and **min** in the output of column names, indicating a hierarchical column index.

If you inspect the row index, you'll get a **MultiIndex** object containing information about the row indices:

```
grouped.index

##
MultiIndex(levels=[['blue', 'green', 'red'], ['pants', 'shirt']],
           labels=[[0, 0, 1, 2], [0, 1, 1, 0]],
           names=['color', 'type'])
```

The column index shows a **MultiIndex** object containing information about the column indices:

```
grouped.columns

#
MultiIndex(levels=[['mass_g', 'price_usd'], ['mean', 'min']],
           labels=[[0, 0, 1, 1], [0, 1, 0, 1]])
```

To perform selection on a dataframe with a MultiIndex, use **loc[]** selection and put indices in parentheses. Here are some examples on **grouped**, which is a dataframe with a two-level row index and a two-level column index. For reference, here is the **grouped** dataframe:

```
grouped

##
            mass_g      price_usd
              mean  min      mean min
color type
blue  pants  200.0  200      40.0  40
      shirt  440.0  440      35.0  35
```

```
green shirt  537.5  395      75.0  50
red   pants  305.0  125      47.5  20
```

To select a first-level (top) column:

```
grouped.loc[:, 'price_usd']

##
           mean  min
color type
blue  pants  40.0   40
      shirt  35.0   35
green shirt  75.0   50
red   pants  47.5   20
```

To select a second-level (bottom) column:

```
grouped.loc[:, ('price_usd', 'min')]

##
color  type
blue   pants    40
       shirt    35
green  shirt    50
red    pants    20
Name: (price_usd, min), dtype: int64
```

To select first-level (left-most) row:

```
grouped.loc['blue', :]

##
      mass_g      price_usd
        mean  min      mean min
type
pants  200.0  200      40.0  40
shirt  440.0  440      35.0  35
```

To select a bottom-level (right-most) row:

```
grouped.loc[('green', 'shirt'), :]

##
mass_g     mean    537.5
           min     395.0
price_usd  mean     75.0
           min      50.0
Name: (green, shirt), dtype: float64
```

And you can even select individual values:

```
grouped.loc[('blue', 'shirt'), ('mass_g', 'mean')]

##

440.0
```

If you want to remove the row MultiIndex from a groupby result, include **as_index=False** as a parameter to your **groupby()** statement:

```
clothes.groupby(['color', 'type'], as_index=False).mean()

##
  color   type  mass_g  price_usd
0  blue  pants   200.0       40.0
1  blue  shirt   440.0       35.0
2 green  shirt   537.5       75.0
3   red  pants   305.0       47.5
```

Notice how **color** and **type** are no longer row indices, but named columns. The row indices are the standard enumeration beginning from zero.

Again, you will not be expected to do any complex manipulations of hierarchically indexed data in this course, but it's helpful to have a basic

understanding of how MultIndex works, especially because **groupby()** manipulations typically result in a MultiIndex dataframe by default.

Key takeaways

groupby() will be an essential function in your work as a data professional, as it enables efficient combining and analysis of data. Similarly, **agg()** will help you apply multiple functions dynamically across a specified axis of a dataframe. Either on their own or when used together, these tools give data professionals deep access to data and help bring about successful projects.

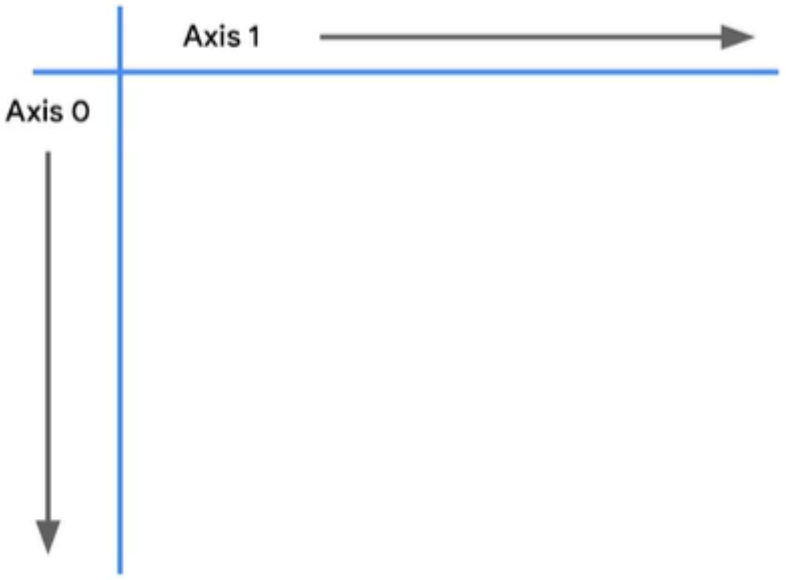
Merging and joining data

pandas functions

- concat()
    - A pandas function that combines data either by adding it horizontally as new columns for existing rows, or vertically as new rows for existing columns
- merge()

concat()

- A pandas function that combines data either by adding it horizontally as new columns for existing rows, or vertically as new rows for existing columns



Pandas has a specific way to indicate which way we want the data to be concatenated. We do this by referring to axes.

In fact, many pandas and NumPy functions have an "axis" keyword so you can specify whether you want to apply the function across rows or down columns.

The two axes of a dataframe are zero, which runs vertically over rows; and one, which runs horizontally across columns.

```
df = pd.concat([list of dataframe that we want to concat], axis = ?)

example:
df3 = pd.concat([df1, df2], axis = 0)
# axis = 0, cuz we want to concat it vertically,
#In other words, we want to add new data by extending axis zero, the vertical
```

[16]:
```
data1 = {'planet': ['Mercury', 'Venus', 'Earth', 'Mars'],
         'radius_km': [2440, 6052, 6371, 3390],
         'moons': [0, 0, 1, 2]
        }
df1 = pd.DataFrame(data = data1)
df1
```

[16]:

	planet	radius_km	moons
0	Mercury	2440	0
1	Venus	6052	0
2	Earth	6371	1
3	Mars	3390	2

[17]:
```
data2 = {'planet': ['Jupiter', 'Saturn', 'Uranus', 'Neptune'],
         'radius_km': [69911, 58232, 25362, 24622],
         'moons': [80, 83, 27, 14],
        }

df2 = pd.DataFrame(data = data2)
df2
```

[17]:

	planet	radius_km	moons
0	Jupiter	69911	80
1	Saturn	58232	83
2	Uranus	25362	27
3	Neptune	24622	14

[18]:
```
df3 = pd.concat([df1, df2], axis = 0)
df3
```

[18]:

	planet	radius_km	moons
0	Mercury	2440	0
1	Venus	6052	0
2	Earth	6371	1
3	Mars	3390	2
0	Jupiter	69911	80
1	Saturn	58232	83
2	Uranus	25362	27
3	Neptune	24622	14

[19]:
```
df3 = df3.reset_index(drop = True)
df3
```

[19]:

	planet	radius_km	moons
0	Mercury	2440	0
1	Venus	6052	0
2	Earth	6371	1
3	Mars	3390	2
4	Jupiter	69911	80
5	Saturn	58232	83
6	Uranus	25362	27
7	Neptune	24622	14

- We include the "drop equals true" argument because otherwise a new index column will be added to the dataframe, which we don't want in this case.

If you want to add data horizontally, consider the merge function

Merge()

- A pandas function that joins two dataframes together; it only combines data by extending along axis one horizontally

keys

- The shared points of reference between different dataframes — what to match on

```
inner = pd.merge(df3, df4, on = 'planet', how = 'inner')
inner

outer = pd.merge(df3, df4, on = 'planet', how = 'outer')
outer

left = pd.merge(df3, df4, on = 'planet', how = 'left')
left

right = pd.merge(df3, df4, on = 'planet', how = 'right')
right
```